Keeping a project organised

Why organise your work?

By now you probably have scripts, downloaded data and outputs scattered around your computer.

```
setwd("C:/Users/ewan/Desktop/R course")
dat <- read_csv("~/Downloads/hes (1).csv")
```

Why organise your work?

By now you probably have scripts, downloaded data and outputs scattered around your computer.

```
setwd("C:/Users/ewan/Desktop/R course")
dat <- read_csv("~/Downloads/hes (1).csv")
```

This works today, on your computer. But it breaks when:

- you move or rename a folder;
- you send the script to a colleague;
- you run it on another computer (or a secure server);
- you come back to it in six months.

Why organise your work?

By now you probably have scripts, downloaded data and outputs scattered around your computer.

```
setwd("C:/Users/ewan/Desktop/R course")
dat <- read_csv("~/Downloads/hes (1).csv")
```

This works today, on your computer. But it breaks when:

- you move or rename a folder;
- you send the script to a colleague;
- you run it on another computer (or a secure server);
- you come back to it in six months.

Aim: a project folder that works anywhere, for anyone.

RStudio projects

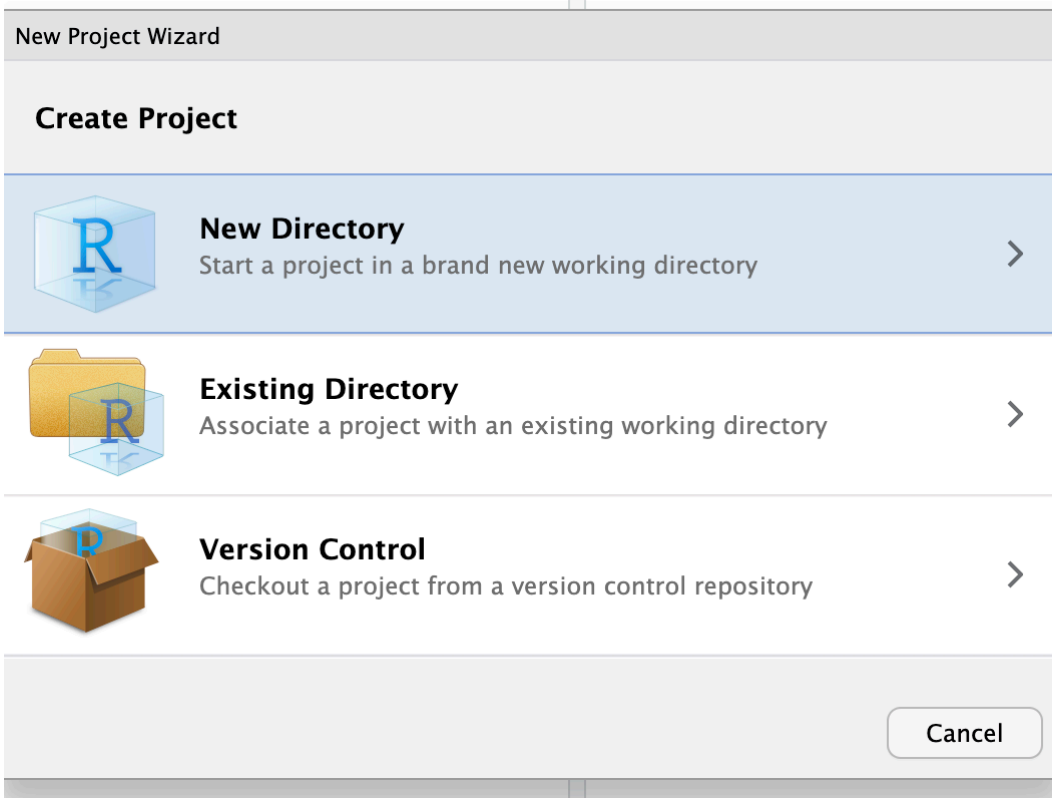
An RStudio project is a folder with an `.Rproj` file in it. Opening the project:

- sets the working directory to the project folder, so files can be found relative to it;
- keeps a separate history and set of open scripts for each project;
- makes it easy to switch between pieces of work (menu in the top right of RStudio).

To open a project, double-click the `.Rproj` file, or use **File** → **Open Project...** in RStudio.

Tip: in **Tools** → **Global Options** → **General**, untick "Restore .RData into workspace at startup" and set "Save workspace to .RData on exit" to "Never". Your scripts, not your workspace, are the record of your work.

Creating a project



1. File → New Project...
2. Choose **New Directory**, then **New Project**.
3. Give it a name (e.g. `hes-practical`) and choose where to save it.

Use **Existing Directory** to turn a folder you already have into a project.

A suggested folder structure

```
hes-practical/
├── hes-practical.Rproj
├── data/
│   ├── hes_patients.csv
│   ├── hes_admissions.csv
│   ├── hes_episodes.csv
│   └── hes_outcomes.csv
├── scripts/
│   ├── 01-import.R
│   ├── 02-clean.R
│   └── 03-analysis.R
└── outputs/
    └── provider-summary.csv
```

`data/`

Raw data, exactly as you received it. Never edit by hand: do all cleaning in code.

`scripts/`

Numbered in the order they run.

`outputs/`

Everything your scripts create. You could delete it and re-run the scripts to get it all back.

Absolute and relative paths

An absolute path starts from the top of *your* computer:

```
read_csv("C:/Users/ewan/hes_admissions.csv")
```

Absolute and relative paths

An absolute path starts from the top of *your* computer:

```
read_csv("C:/Users/ewan/hes_admissions.csv")
```

A relative path starts from the working directory, which is the project folder:

```
read_csv("data/hes_admissions.csv")
```

Absolute and relative paths

An absolute path starts from the top of *your* computer:

```
read_csv("C:/Users/ewan/hes_admissions.csv")
```

A relative path starts from the working directory, which is the project folder:

```
read_csv("data/hes_admissions.csv")
```

Why not just use `setwd()`?

- The path inside it only exists on your computer.
- It changes the working directory for everything that runs afterwards.
- Quarto documents always run from their own folder, whatever you set.

Building paths with `here()`

The `here` package finds the project folder (the one with the `.Rproj` file) and builds paths from there.

```
library(here)

here()
[1] "/Users/ewan/projects/hes-practical"

read_csv(here("data", "hes_admissions.csv"))

write_csv(provider_summary,
  here("outputs", "provider-summary.csv"))
```

Building paths with `here()`

The `here` package finds the project folder (the one with the `.Rproj` file) and builds paths from there.

```
library(here)

here()
[1] "/Users/ewan/projects/hes-practical"

read_csv(here("data", "hes_admissions.csv"))

write_csv(provider_summary,
  here("outputs", "provider-summary.csv"))
```

This works in a script in `scripts/`, in a Quarto document in any subfolder, and on Windows, Mac or Linux.

Checklist

1. One RStudio project for each piece of work.
2. Raw data in `data/`, never edited by hand.
3. Scripts in `scripts/`, numbered in the order they run.
4. Anything a script creates goes in `outputs/`.
5. Paths built with **`here()`**: no `setwd()`, no `~/Downloads`.
6. Don't save your workspace. Restart R and re-run your scripts.

Practical 3: set up your project

1. Create an RStudio project called `hes-practical`, with folders `data`, `scripts` and `outputs`.
2. Download the four HES files into `data/`.
3. In `scripts/01-import.R`, import them with `read_csv()` and `here()`.
4. Then carry on with Part 1 of the practical.